

PROYECTO INTERMODULAR GLAMURCLUB

Documentación Técnica Transversal del Sistema

Integrantes: Adrián Becerra y Jose Juan Alemany

Aplicación: Ecosistema Global Glamur Club

Identificador: Grupo 04 (XX = 04)

Este documento proporciona una visión global e integrada del ecosistema de la aplicación Glamur Club, detallando la relación entre el frontend desacoplado, el backend estructurado como API REST, la persistencia de datos, las redes virtuales y las políticas de automatización para la puesta en producción del sistema.



1. Gestión de Dominios y Configuración DNS (Part 1)

El acceso a la infraestructura en producción está centralizado bajo la zona de dominio delegada de la institución. Para este proyecto, se define formalmente la zona asignada al grupo de trabajo:

Zona de trabajo: `projecte04.ddaw.es`

(Donde `04` representa el identificador del grupo de trabajo especificado).

Registros de Zona Implementados

Para garantizar la separación de responsabilidades y permitir el direccionamiento independiente de los servicios sin interferencias, se configuran los siguientes registros de tipo A y CNAME apuntando hacia el punto único de entrada del tráfico en la nube (IP Elástica asignada al servidor de producción en AWS):

Subdominio	Tipo	Destino / Valor	Propósito
projecte04.ddaw.es	A	IP_ELASTICA_AWS	Acceso principal a la SPA (Frontend en Vue.js).
api.projecte04.ddaw.es	A	IP_ELASTICA_AWS	Punto de enlace exclusivo para la API REST (Backend en Laravel).
www.projecte04.ddaw.es	CNAME	projecte04.ddaw.es	Redirección estándar de cortesía web.



2. Entorno de Desarrollo Local con Docker (Part 2)

Con el fin de asegurar un entorno reproducible, homogéneo e independiente del sistema operativo de los desarrolladores, el proyecto está completamente containerizado. Esto mitiga de forma absoluta el problema clásico de *"en mi máquina funciona"*, unificando las dependencias del equipo de desarrollo.

2.1. Contenedor del Frontend (Vue.js)

Ubicado en el directorio `./frontend/Dockerfile`, este archivo define una construcción ligera basada en Node.js de infraestructura para el desarrollo reactivo de la interfaz de usuario.

```
./frontend/Dockerfile
```

```
FROM node:20-alpine

WORKDIR /app

COPY package*.json ./
RUN npm install

COPY . .

EXPOSE 5173

CMD ["npm", "run", "dev", "--", "--host", "0.0.0.0"]
```

2.2. Contenedor del Backend (Laravel API)

Ubicado en el directorio `./laravel/Dockerfile`, incorpora todas las extensiones nativas necesarias para dar soporte a Eloquent ORM, la conexión segura con MySQL, cifrado de datos y la manipulación de paquetes de backend vía Composer.

./laravel/Dockerfile

```
FROM php:8.2-cli

RUN apt-get update && apt-get install -y      libzip-dev zip unzip git curl libonig-
dev default-mysql-client      && docker-php-ext-install pdo_mysql zip mbstring

COPY --from=composer:latest /usr/bin/composer /usr/bin/composer

WORKDIR /app

COPY . .

EXPOSE 8000

CMD ["php", "artisan", "serve", "--host=0.0.0.0", "--port=8000"]
```

 2.3. Orquestador Maestro (docker-compose.yml)

Ubicado en la raíz del proyecto integrador, coordina el ciclo de vida de los servicios de aplicación y la persistencia de la base de datos MySQL mediante volúmenes nombrados y aislamiento de red local.

```
./docker-compose.yml
```

```
version: '3.8'

services:
  db:
    image: mysql:8.0
    container_name: glamur_db
    restart: unless-stopped
    environment:
      MYSQL_DATABASE: glamur_db
      MYSQL_ROOT_PASSWORD: root
    ports:
      - "3306:3306"
    volumes:
      - db_data:/var/lib/mysql

  backend:
    build:
      context: ./laravel
      dockerfile: Dockerfile
    container_name: glamur_backend
    ports:
      - "8000:8000"
    volumes:
      - ./laravel:/app
    depends_on:
      - db

  frontend:
    build:
      context: ./frontend
      dockerfile: Dockerfile
    container_name: glamur_frontend
    ports:
      - "5173:5173"
    volumes:
      - ./frontend:/app
      - /app/node_modules

volumes:
  db_data:
```



3. Entorno de Producción e Integración Continua (Part 3)



3.1. Flujo de CI/CD (GitHub Actions)

La automatización se gestiona mediante pipelines independientes y desacoplados basados en eventos de integración en la rama activa de desarrollo `sprint5-6`. Esto garantiza el aislamiento estricto de entornos, de modo que cambios exclusivos en el diseño de la interfaz no requieran la reinstalación ni causen interrupciones en la lógica de negocio del backend.

```

[ Desarrollador ] ---> git push origin sprint5-6
                        |
                        +-----+-----+
                        |               |
                    (laravel/**)      (frontend/**)
                        v               v
[ Pipeline Backend ]      [ Pipeline Frontend ]
- Install Composer        - Install NPM packages
- Setup Environment        - Run Linter / Vitest
- Run Migrations (Test Environment)
- Simulate Deploy to AWS EC2    - Compilation (NPM Build)
                                - Simulate Deploy to Web Root

```



3.2. Seguridad HTTPS y Terminación SSL (Let's Encrypt)

En el entorno de producción real, el tráfico está protegido mediante certificados SSL/TLS válidos emitidos de forma gratuita y automatizada por la entidad certificadora **Let's Encrypt**.

- **Mecanismo:** Un servidor web Nginx actúa como proxy inverso escuchando activamente en los puertos 80 y 443 de la máquina de producción.
- **Aislamiento:** Nginx intercepta las peticiones externas, valida el certificado seguro de forma transparente y redirige el tráfico internamente hacia los contenedores de aplicación correspondientes. De esta manera, se evita exponer directamente los servidores internos de Node.js o el CLI de PHP a la red pública, blindando la arquitectura.



4. Normas de Contribución y Flujo de Trabajo (Part 5)

Para mantener la calidad del código y la coherencia estilística a lo largo de todas las fases del proyecto, el equipo adopta de forma estricta las siguientes políticas organizacionales:

- **Estrategia de Ramas:** Se emplea un flujo derivado de *GitFlow*. La rama `main` contiene exclusivamente versiones estables de producción. El desarrollo de los últimos entregables se centraliza en la rama unificada `sprint5-6`, desde la cual se realizan pruebas cruzadas exhaustivas antes de integrarse de forma definitiva.
- **Estilo de Código (Code Style):** El frontend sigue estrictamente las reglas recomendadas de *Vue ESLint*. El backend en Laravel se adhiere a los estándares internacionales de codificación *PSR-12* para garantizar una sintaxis PHP limpia y homogénea.
- **Mensajes de Commit:** Se utiliza un patrón semántico descriptivo para el control riguroso del historial de versiones. Ejemplos de uso obligatorio y estructurado para el equipo:
 - `[FEAT]` Para la incorporación de nuevas funcionalidades (ej. *Google Login*).
 - `[FIX]` Para la resolución de bugs detectados (ej. *Formato de valoraciones*).
 - `[DOCS]` Para la redacción y actualización de guías o manuales de usuario.

5. Usuarios de Prueba para Validación (Part 5)

Con el fin de que los evaluadores y el tribunal puedan verificar de forma exhaustiva los mecanismos de control de accesos basados en roles granularizados y los permisos de la aplicación, se proporcionan las siguientes cuentas preconfiguradas mediante los seeders del sistema (estos datos están limpios de contraseñas personales o corporativas reales):

Rol Administrador *(Control Total del Catálogo)*

Correo: `admin@glamur.com`

Contraseña: `password123`

Rol Moderador *(Gestión de Comentarios y Reseñas)*

Correo: `moderador@glamur.com`

Contraseña: `password123`

Rol Cliente Estándar *(Acceso General y Navegación)*

Correo: `cliente@glamur.com`

Contraseña: `password123`